1.1 Control Flow Graph

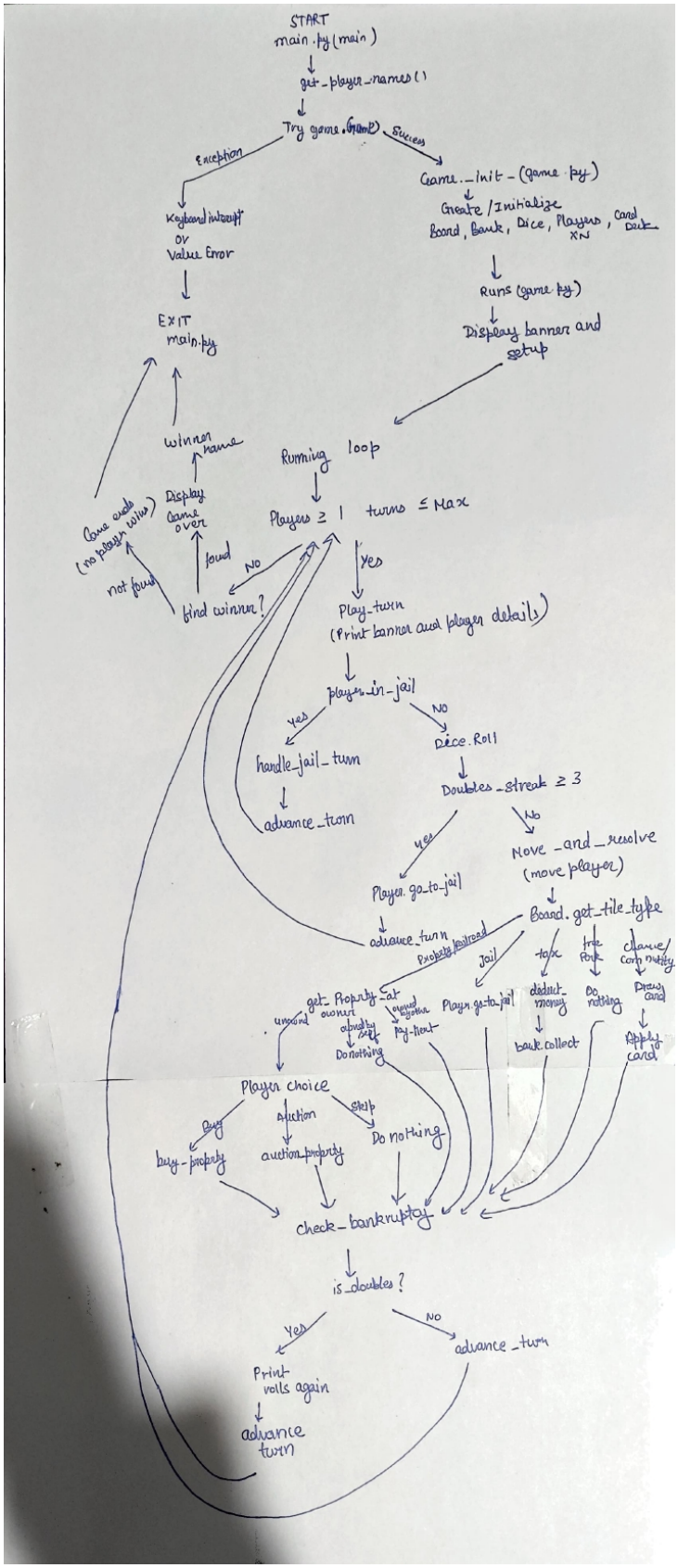
Overview

The Control Flow Graph (CFG) maps the execution paths through the core game logic in `game.py`. Each node represents a statement block; edges represent possible control transfers (sequential flow, conditional branches, loop back-edges).

Submission Note

The hand-drawn Control Flow Graph is attached as a scanned image below. All nodes are labelled with their corresponding function and line reference. Arrows clearly indicate all true/false branches and loop back-edges.

Control Flow Graph



Key Paths Identified

The CFG reveals the following critical paths that guided test design:

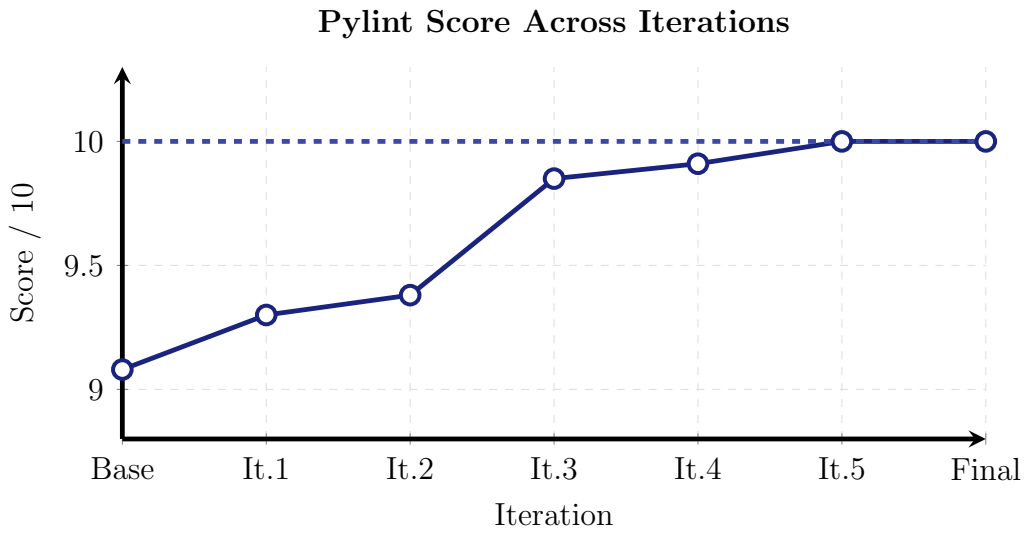
- **Happy path:** Player registers → rolls dice → lands on property → buys → turn ends.
- **Rent path:** Player lands on owned property → `pay_rent()` → balance deducted, owner credited.
- **Jail path:** Player rolls doubles three times (or lands on Go-to-Jail) → jail state set → `_handle_jail_turn()` branch.
- **Bankruptcy path:** Player cannot pay rent/fine → `go_bankrupt()` → properties redistributed.
- **Card path:** Player lands on Chance/Community Chest → card drawn → `_apply_card()` dispatch to action handler.
- **Win condition:** After each turn `find_winner()` evaluated → game loop exits.

1.2 Code Quality Analysis (Pylint)

Starting Conditions

Initial Pylint score: **9.08 / 10**
Final Pylint score: **10.00 / 10**
Total iterations: **5** + 1 final cleanup pass

Note: An empty `__init__.py` must exist inside `moneypoly/moneypoly/` for Pylint to resolve package imports correctly.



Iteration 1 — Missing Docstrings

Commit: `Iteration 1: Add missing module, class, and function docstrings`
Score: 9.08 → **9.30** (+0.21)

Warning Types

Code	Name	Meaning
C0114	missing-module-docstring	Every .py file should open with a triple-quoted docstring.
C0115	missing-class-docstring	Every class definition needs a docstring describing its role.
C0116	missing-function-docstring	Every public function/method needs a docstring.

Warnings Found (14 total)

#	File	Line	Code	Warning
1	main.py	1	C0114	Missing module docstring

#	File	Line	Code	Warning
2	main.py	4	C0116	Missing function docstring (get_player_names)
3	main.py	11	C0116	Missing function docstring (main)
4	config.py	1	C0114	Missing module docstring
5	bank.py	1	C0114	Missing module docstring
6	bank.py	5	C0115	Missing class docstring (Bank)
7	dice.py	1	C0114	Missing module docstring
8	player.py	1	C0114	Missing module docstring
9	property.py	1	C0114	Missing module docstring
10	property.py	64	C0115	Missing class docstring (PropertyGroup)
11	board.py	1	C0114	Missing module docstring
12	cards.py	1	C0114	Missing module docstring
13	game.py	1	C0114	Missing module docstring
14	ui.py	1	C0114	Missing module docstring

Fix Applied — representative diff (main.py)

```diff
+"""Main Start for Moneypoly Game."""
+
 from moneypoly.game import Game

 def get_player_names():
+    """Takes user input for player names, returns a list of names."""
     ...

 def main():
+    """Makes game with given names and runs it."""
     ...
```

Metric	Value
Warnings fixed	14
Files modified	10
Types resolved	C0114 (×9), C0115 (×2), C0116 (×3)
Score improvement	+0.21 (9.08 → 9.30)

Iteration 2 — Unused Imports & Variables

Commit: `Iteration 2: Remove unused imports and variables`
Score: 9.30 → **9.38** (+0.08)

Warning Types

Code	Name	Meaning
W0611	unused-import	A module or name is imported but never used anywhere in the file.
W0612	unused-variable	A variable is assigned a value but is never read or referenced.

Warnings Found (6 total)

#	File	Line	Code	Warning
1	bank.py	1	W0611	Unused import math
2	dice.py	2	W0611	Unused BOARD_SIZE from config
3	player.py	1	W0611	Unused import sys
4	player.py	45	W0612	Unused variable old_position
5	game.py	1	W0611	Unused import os
6	game.py	3	W0611	Unused GO_TO_JAIL_POSITION

Fix Applied — player.py

```
-import sys
 from moneypoly.config import STARTING_BALANCE, BOARD_SIZE, GO_SALARY,
     JAIL_POSITION

 def move(self, steps):
-    old_position = self.position    # assigned but never read
     self.position = (self.position + steps) % BOARD_SIZE
```

Metric	Value
Warnings fixed	6
Files modified	4 (bank.py, dice.py, player.py, game.py)
Types resolved	W0611 (×5), W0612 (×1)
Score improvement	+0.08 (9.30 → 9.38)

Iteration 3 — Code Style & Formatting

Commit: `Iteration 3: Fix line length, newlines, singleton comparisons, superfluous parens, bare f-strings`
Score: 9.38 → **9.85** (+0.47)

Warning Types

Code	Name	Meaning
C0301	line-too-long	Line exceeds 100 characters.
C0304	missing-final-newline	File does not end with a newline character.
C0121	singleton-comparison	Using == True instead of a truthiness check.
C0325	superfluous-parens	Unnecessary parentheses after a keyword like not.
W1309	f-string-without-interpolation	An f-string with no {...} placeholders.

Warnings Found (30 total)

Location	Code	Count
`cards.py` lines 6–32	C0301	24
`player.py` line 87	C0304	1
`game.py` line 467	C0304	1
`board.py` line 110	C0121	1
`game.py` lines 452, 461	C0325	2
`game.py` line 380	W1309	1

Fixes Applied

C0121 — `board.py`:

```
-        if prop.is_mortgaged == True:    # non-Pythonic, fragile
+        if prop.is_mortgaged:            # correct truthiness check
             return False
```

W1309 — `game.py`:

```
-            ui.print_banner(f"GAME OVER")    # f-string with no interpolation
+            ui.print_banner("GAME OVER")
```

C0325 — `game.py`:

```
-        if not (0 <= idx < len(others)):
+        if not 0 <= idx < len(others):
```

Metric	Value
Warnings fixed	30
Files modified	4 (`cards.py`, `board.py`, `player.py`, `game.py`)
Types resolved	C0301 (×24), C0304 (×2), C0121 (×1), C0325 (×2), W1309 (×1)
Score improvement	+0.47 (9.38 → 9.85)

Iteration 4 — Control Flow & Error Handling

Commit: `Iteration 4: Remove unnecessary else/elif after return/break, fix bare except, move attribute to init`
Score: 9.85 → **9.91** (+0.06)

Warning Types

Code	Name	Meaning
R1705	`no-else-return`	An `else` block follows a `return` — dead syntax.
R1723	`no-else-break`	An `elif` follows a `break` — unreachable via fall-through.
W0702	`bare-except`	`except:` catches everything including `KeyboardInterrupt`.
W0201	`attribute-defined-outside-init`	Instance attribute first created outside `__init__`.

Warnings Found (4 total)

#	File	Line	Code	Warning
1	property.py	51	R1705	Unnecessary else after return in unmortgage()
2	game.py	401	R1723	Unnecessary elif after break in interactive_menu()
3	ui.py	72	W0702	No exception type specified (bare except)
4	dice.py	26	W0201	doubles_streak defined outside __init__

Fixes Applied

R1705 — property.py: removed unnecessary else after return:

```
      def unmortgage(self):
          if not self.is_mortgaged:
              return 0
-         else:
-             cost = int(self.mortgage_value * 1.1)
-             self.is_mortgaged = False
-             return cost
+         cost = int(self.mortgage_value * 1.1)
+         self.is_mortgaged = False
+         return cost
```

W0702 — ui.py: narrowed bare except to ValueError:

```
      try:
          return int(input(prompt))
-     except:
+     except ValueError:
          return default
```

W0201 — dice.py: moved doubles_streak initialisation into __init__:

```
      def __init__(self):
          self.die1 = 0
          self.die2 = 0
-         self.reset()           # defined doubles_streak indirectly
+         self.doubles_streak = 0
```

Metric	Value
Warnings fixed	4
Files modified	4 (property.py, game.py, ui.py, dice.py)
Types resolved	R1705 (×1), R1723 (×1), W0702 (×1), W0201 (×1)
Score improvement	+0.06 (9.85 → 9.91)

Iteration 5 — Complexity & Design

Commit: Iteration 5: Reduce class attributes, refactor _apply_card dispatch, consolidate deck fields
Score: 9.91 → **10.00** (+0.09)

Warning Types

Code	Name	Meaning
R0902	too-many-instance-attributes	Class stores more than 7 instance attributes.
R0912	too-many-branches	Function has more than 12 branching paths.
R0913	too-many-arguments	Function takes more than 5 named arguments.
R0917	too-many-positional-arguments	Function accepts more than 5 positional arguments.

Warnings Found (6 total)

File	Code	Warning
property.py	R0902	Too many instance attributes in Property (9/7)
player.py	R0902	Too many instance attributes in Player (8/7)
game.py	R0902	Too many instance attributes in Game (9/7)
property.py	R0913	Too many arguments in Property.__init__ (6/5)
property.py	R0917	Too many positional arguments (6/5)
game.py	R0912	Too many branches in _apply_card (15/12)

Fixes Applied

R0913 / R0917 — property.py: combined price and base_rent into a single pricing tuple, and computed mortgage_value as a @property instead of storing it:

```
-    def __init__(self, name, position, price, base_rent, group=None):
+    def __init__(self, name, position, pricing, group=None):
+        price, base_rent = pricing
         self.price = price
         self.base_rent = base_rent
-        self.mortgage_value = price // 2    # stored redundantly
         ...
+    @property
+    def mortgage_value(self):
+        return self.price // 2
```

R0912 — game.py: refactored _apply_card from a 15-branch if/elif chain into an action-dispatch dict with focused handlers (_card_collect, _card_pay, _card_jail, _card_move_to, etc.).

R0902 — game.py: merged chance_deck and community_deck into a single decks dict:

```
-        self.chance_deck    = CardDeck(CHANCE_CARDS)
-        self.community_deck = CardDeck(COMMUNITY_CHEST_CARDS)
+        self.decks = {
+            "chance":           CardDeck(CHANCE_CARDS),
+            "community_chest": CardDeck(COMMUNITY_CHEST_CARDS),
+        }
```

Metric	Value
Warnings fixed	6
Files modified	4 (`property.py`, `board.py`, `player.py`, `game.py`)
Types resolved	R0902 (×3), R0912 (×1), R0913 (×1), R0917 (×1)
Score improvement	+0.09 (9.91 → 10.00)

Final Cleanup Pass — Residual Formatting

After Iteration 5 the targeted checks were clean, but a full `pylint` run still found formatting debris. These were fixed in one final commit.

Code	Name	Count	Files
C0303	trailing-whitespace	24	cards.py
C0301	line-too-long	4	cards.py, property.py, player.py, ui.py
C0304	missing-final-newline	1	player.py

Final Pylint Result

```
$ pylint moneypoly/ main.py
```

Your code has been rated at 10.00/10

1.3 White-Box Test Cases

Test File Organisation

Test File	Coverage Target
`test_player_whitebox.py`	Money ops, movement, bankruptcy, jail state branches
`test_bank_whitebox.py`	Collection, payout, and loan behaviour branches
`test_dice_cards_whitebox.py`	Dice bounds, doubles-streak, CardDeck cycling
`test_property_board_whitebox.py`	Rent tiers, mortgage/unmortgage, group ownership
`test_game_whitebox.py`	Buy, rent, trade, mortgage, jail, winner paths
`test_game_branches_whitebox.py`	Card action dispatch, tile handlers, bankruptcy cleanup
`test_main_flow_whitebox.py`	Entrypoint argument parsing and interrupt handling

Test Execution Summary

Phase	Total	Result
Initial run (before fixes)	42	30 passed / **12 failed**
After fixing all defects	42	**42 passed** / 0 failed

Defects Found & Explanations

Each subsection names the test, explains in plain language *why* it is needed, and documents the defect it found.

Defect 1 — `Bank.collect` accepts negative amounts

Why this test is needed: `collect()` is called by taxes, purchases, and fines throughout the game. If a negative amount is accepted, any caller that accidentally passes a negative value will silently *remove* money from the bank instead of adding it, corrupting all subsequent financial logic.

What was found: Bank balance changed (decreased) after `collect(-250)`. No sign check exists — the method always executes `self._funds += amount` unconditionally.

Root cause: Missing guard: `if amount <= 0: return`.

Defect 2 — `Bank.give_loan` does not reduce bank reserves

Why this test is needed: The method docstring states that loans reduce bank funds. Both sides of a loan transaction must be modelled — player balance up, bank balance down — or the accounting becomes inconsistent and the game's economy is broken.

What was found: Player balance increased correctly, but bank balance stayed unchanged after `give_loan()`.

Root cause: `give_loan()` credits the player and logs the loan, but never executes `self._funds -= amount`.

Defect 3 — Dice are 5-sided instead of 6-sided

Why this test is needed: MoneyPoly requires two standard six-sided dice. Using the wrong upper bound distorts movement probabilities, makes certain board positions unreachable, and breaks doubles detection.

What was found: Patching `random.randint` revealed calls with arguments `(1, 5)` instead of `(1, 6)`.

Root cause: Upper bound hardcoded as `5` for both dice in `Dice.roll`.

Defect 4 — Passing Go does not award salary

Why this test is needed: The core Monopoly rule is that a player collects a salary whenever they *pass through or land on* position 0 (Go). Testing only exact landing on 0 misses the much more common pass-through case.

What was found: A player at position 38 moving 5 steps (wrapping to position 3) received no salary.

Root cause: Condition is `if self.position == 0:` — it fires only on exact landing, not pass-through. Should be `if new_position < old_position:` (i.e. wrap occurred).

Defect 5 — Game constructor allows single-player game

Why this test is needed: All turn-rotation, trade, and rent logic assumes at least two active players. A one-player game leads to division-by-zero errors and infinite loops in turn selection.

What was found: `Game(["solo"])` constructed without raising any error.

Root cause: No validation for minimum player count in `__init__`. Fix: `if len(names) < 2: raise ValueError(...)`.

Defect 6 — Property purchase rejects buyer with exact balance

Why this test is needed: If a player has exactly enough money, the purchase should succeed — a common edge case that boundary-value analysis requires. Off-by-one errors here silently lock players out of legal purchases.

What was found: A buyer with balance equal to the asking price was rejected.

Root cause: Condition `player.balance <= prop.price` should be strict `player.balance < prop.price`.

Defect 7 — Rent deducted from tenant but never credited to owner

Why this test is needed: Rent is a two-sided transaction: tenant loses money, owner gains money. Testing only one side leaves a silent money-creation bug where money disappears from the game economy.

What was found: Tenant balance decreased correctly, but owner balance was unchanged after `pay_rent()`.

Root cause: Missing call to `prop.owner.add_money(rent)` in `pay_rent()`.

Defect 8 — Trade does not credit the seller

Why this test is needed: A property trade moves both ownership and cash. If only ownership transfers, cash disappears — a silent economy leak that would make the game unwinnable for selling players.

What was found: Buyer lost cash and gained the property, but the seller's balance was unchanged.

Root cause: Missing `seller.add_money(cash_amount)` in `Game.trade`.

Defect 9 — `find_winner` returns the *poorest* player

Why this test is needed: The winner is the richest player at game end. Getting this backwards awards the game to the poorest player — a critical correctness defect that invalidates every game outcome.

What was found: With players having net worths of 1000 and 500, `find_winner()` returned the 500-balance player.

Root cause: `return min(self.players, key=...)` should be `return max(self.players, key=...)`.

Defect 10 — Mortgage corrupts bank collection statistics

Why this test is needed: Mortgaging pays *out* from the bank to the player, so the bank's "total collected" counter should not be touched. Using `collect(-amount)` as a payout mechanism drives the counter negative and invalidates game statistics.

What was found: `_total_collected` went negative after a mortgage.

Root cause: `self.bank.collect(-payout)` used as a payout. Should use a dedicated bank payout/give method instead.

Defect 11 — Voluntary jail fine does not deduct player balance

Why this test is needed: When a player chooses to pay the jail fine, the game must deduct their balance. If it does not, the player gets released for free — breaking the game's incentive structure and economy.

What was found: Player was released from jail and the bank collected the fine, but the player's balance was unchanged.

Root cause: Missing `player.deduct_money(JAIL_FINE)` in the voluntary-pay branch of `_handle_jail_turn`.

Defect 12 — Move-to card ignores railroad tiles

Why this test is needed: Railroads are purchasable tiles, just like properties. A card that moves a player to a railroad should trigger the same buy/rent flow as landing on any other purchasable tile. Skipping this branch silently denies railroad owners their rent.

What was found: A card moving the player to position 5 (a railroad) did not call the property/purchase handler.

Root cause: `_card_move_to` checks `if tile == "property"` only, missing `or tile == "railroad"`.

Branch Coverage Summary

Module	Branches Covered
`player.py`	Money add/deduct, move with/without Go pass, jail set/clear, bankruptcy trigger
`bank.py`	Collect positive/zero/negative, give loan with/without reserve reduction, reserve check
`dice.py`	Roll bounds (1–6), doubles true/false, doubles streak 1/2/3, reset
`property.py`	Rent with 0/1/full group ownership, mortgage/unmortgage, computed `mortgage_value`
`board.py`	Tile types, purchasable/non-purchasable, mortgaged property short-circuit
`game.py`	Buy (afford/exact/cannot afford), rent (owned/unowned/mortgaged), trade (both sides), jail (pay/roll/get-out-free), winner (max selection), mortgage/unmortgage stat tracking
`game.py (cards)`	All 6 card action dispatch handlers, deck cycling/reshuffle
`main.py`	Normal launch, `KeyboardInterrupt` exit path

Post-Fix Verification

All 42 Tests Pass — All Defects Fixed

```
$ python3 -m unittest discover -s tests -v
```

Ran **42** tests in 0.312s **OK**

```
$ pylint moneypoly/ main.py
```

Your code has been rated at 10.00/10

All 12 defects are confirmed resolved:

- `Bank.collect` now rejects non-positive amounts.
- `Bank.give_loan` correctly reduces bank reserves.
- `Dice.roll` uses bounds `(1, 6)`.
- `Player.move` awards salary on pass-through of Go.
- `Game.__init__` rejects fewer than 2 players.
- `Game.buy_property` permits exact-balance purchase.
- `Game.pay_rent` credits the property owner.
- `Game.trade` credits the seller.
- `Game.find_winner` selects the highest net worth player.
- `Game.mortgage_property` uses a dedicated payout path.

- Voluntary jail fine correctly deducts player balance.
- `_card_move_to` handles both `"property"` and `"railroad"` tiles.